DATA SCIENCE & AI LAB · GROUP 1

# Multimodal Damage Assessment for Insurance Claims

Milestone 3 — Model Architecture Selection & Pipeline Design

Satyajeet Kumar
Anuj Gautam
Pranab Kumar Manna
Venkata Siva Kamal Guddanti
Harsh Pal

# Recap & Objectives

Milestone 1 defined the problem: automate the first-pass review of vehicle-damage insurance claims — detect damage, retrieve the governing policy clauses, and generate a structured preliminary report. Milestone 2 delivered the data foundation.

**13,655**

deduplicated VehiDE images

**32,672**

retained damage instances

**6.59 : 1**

scratch : shattered_glass imbalance

**185**

policy clauses indexed (5 docs)

## MILESTONE 3 OBJECTIVES

1. **Select & justify**

   a model for every pipeline stage — Damage, Severity, Policy retrieval, Report generation — against alternatives and empirical evidence.

2. **Design the pipeline**

   end to end: user input to rendered report, including error handling and the human-escalation path.

*Contribution framing: not a novel model architecture, but a modular, independently-debuggable, cost-appropriate pipeline — the smallest, most measurable, most deployable model at every stage.*

# Four Agents, One Shared Context Bundle

**Damage Agent**

**Severity Agent**

**Policy Agent**

**Report Agent**

*Sequential pipeline today each stage is a plain function call over a shared, schema-defined bundle. Wrapping the stages as LangGraph nodes is planned for Milestone 4, not yet implemented.*

### No orchestration framework yet

Plain sequential Python calls; each stage is independently testable against the shared schema.

### Escalation is a flag, not a gate

Low confidence / missing coverage flows through — the Report Agent self-reports escalate_to_human in its own output.

### Policy is selected, not inferred

A 315-case exhaustive census proved damage profile cannot recover the applicable policy (Section 5.3).

# One Model per Stage — Selected & Justified

| Module | Model Selected | Type | Role |
|---|---|---|---|
| Damage Agent | YOLO11m (Ultralytics) | Fine-tuned | Bounding-box detection, 6 classes |
| Severity Agent | Calibrated area-ratio proxy | Rule-based | Minor / Moderate / Severe |
| Policy — embedding | all-MiniLM-L6-v2 | Frozen, pre-trained | 384-d dense query/chunk embedding |
| Policy — sparse | TF-IDF (sklearn) | Fit on corpus | Lexical retrieval, fused via RRF |
| Policy — vector store | ChromaDB | Infrastructure | Persistent ANN index, doc-scoped |
| Report Agent | llama-3.3-70b-versatile<br>+ openai/gpt-oss-20b | Prompted only (Groq) | Structured report generation |

*No module is trained end-to-end except the Damage Agent — the retrieval stack uses frozen/fit-only components, and the Report Agent is prompted, not fine-tuned (Milestone 1 scope).*

# YOLO11m — Single-Stage Detector, Three Parts

### 1 Backbone

CSP-style feature extractor — C3k2 blocks (replacing YOLOv8's C2f), producing multi-scale feature maps.

### 2 Neck

PAN-FPN feature fusion + C2PSA partial self-attention at the deepest stage — improves small-object context, directly relevant given a 0.033 median normalised bbox area.

### 3 Head

Decoupled detection head — separate classification and box-regression branches, producing per-instance boxes and confidence.

**SCALE SELECTED: `m` (MEDIUM)**

**20.1M**

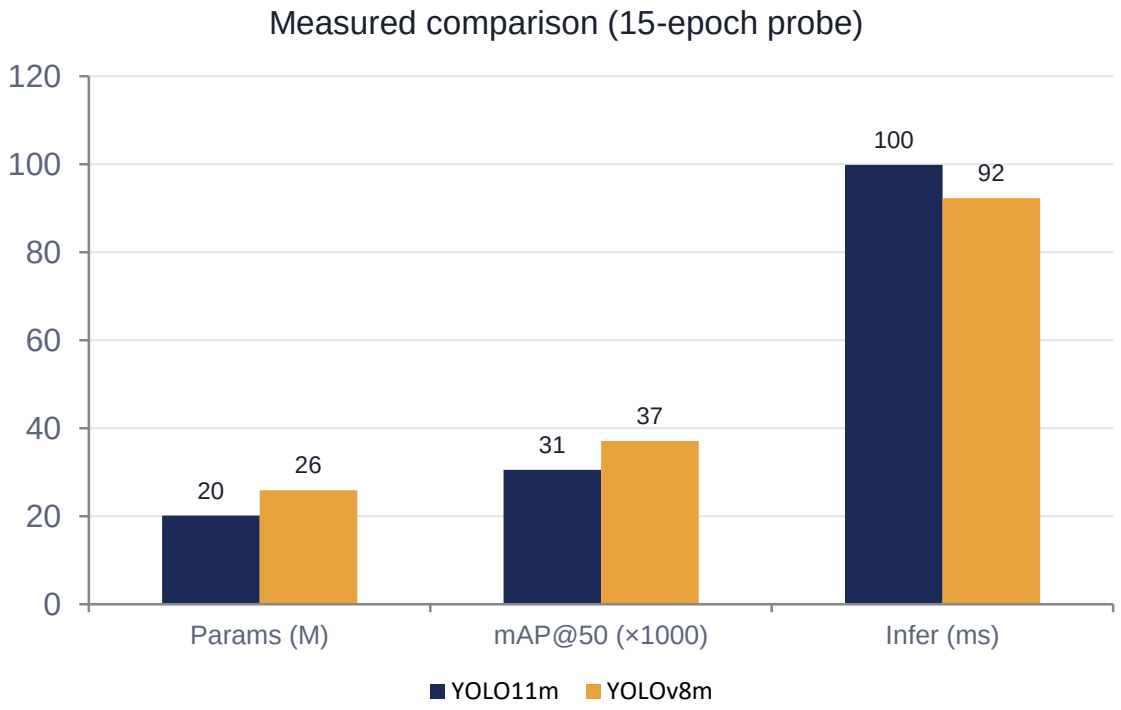
parameters — measured

- n / s variants — faster, lower accuracy
- l / x variants — too slow/large for CPU-basic HF Spaces inference target
- m — the balance point for this deployment target

*Segmentation head (-seg) is not used: VehiDE preprocessing produced bounding-box labels only, so plain detection is the correct fit, not a fallback.*

# YOLO11m vs. YOLOv8m

Both architectures trained on an identical 3,000-image subsample, escalated 3 → 5 → 15 epochs, under identical hyperparameters.




## VERDICT: STATISTICAL TIE ON ACCURACY

| | |
|---|---|
| **mAP@50 delta** | 0.0066 (< 0.02 tie-break threshold) |
| **mAP@50-95** | 0.0046 vs 0.0081 |
| **Training time / epoch** | 1,370.8s vs 1,265.8s |
| **Peak inference memory** | 1.45 GB vs 1.44 GB |

Decision: YOLO11m selected on the pre-registered tie-break — C3k2/C2PSA attention blocks are reported to aid small-object detection, directly relevant given this dataset's tiny bounding boxes. YOLOv8m retained as the Milestone 4 baseline comparison at full scale.

# Hybrid Dense + Sparse Retrieval, Fused via RRF

EMBEDDING

## all-MiniLM-L6-v2

22.7M params, 384-d, frozen. 1.00 vs 0.94 Precision@3 against BGE-small on this corpus.

SPARSE

## TF-IDF (sklearn)

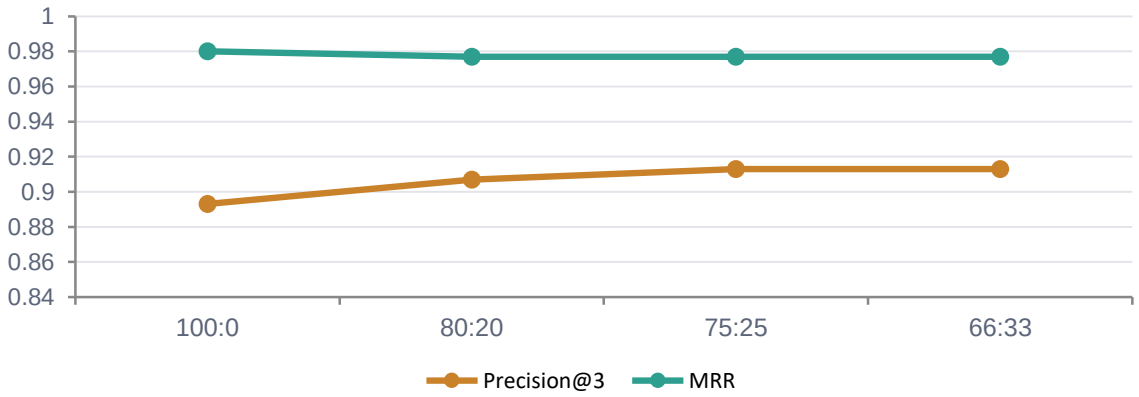
Fit once on the 185-chunk corpus. Fixed the one dense-only zero-hit failure — a query mentioning "window" needing lexical, not semantic, weight.

VECTOR STORE

## ChromaDB

FAISS is ~50-60× faster in raw query latency — not operationally meaningful at 185 chunks. ChromaDB wins on metadata filtering (doc-scoping) + persistence.

### DENSE : SPARSE WEIGHT SWEEP (50-INCIDENT EVAL)




### 75 : 25 SELECTED

75:25 and 66:33 tie on aggregate metrics — 75:25 kept as the more conservative choice, weighting the already-stronger dense signal more heavily. Zero zero-hit incidents at every ratio tested.

# Policy Is Selected, Not Inferred

### REJECTED: AUTO-SELECT FROM DAMAGE

**315**

exhaustive cases
(63 class subsets × 5 policies)

**0.200**

top-1 accuracy — identical to
random chance (1/5 policies)

Why it fails structurally, not just statistically: every policy covers all 6 damage classes, and everything that differs (insurer identity, depreciation tier, add-ons) is a contract fact, not a damage fact. More data would not fix this.

**→ The claimant explicitly selects a policy from a described catalog (Stage 1). Retrieval below is scoped to that choice.**

### STAGE 2 — TWO-PASS, DOC-SCOPED RETRIEVAL

Per detected damage class, two separate queries scoped to the selected policy's doc_id:

**Coverage query**

up to 5 coverage/definition chunks

**Exclusion query**

up to 5 exclusion/sub-limit/condition chunks

*Why two passes: a single mixed query tends to rank the coverage clause highest and bury the exclusion that actually caps or voids it — a report that only ever sees coverage will over-approve.*

09 · REPORT AGENT

# Two Open-Weight Models, Run Head-to-Head via Groq

### `llama-3.3-70b-versatile`

**70B**

Dense, strong general reasoning

### `openai/gpt-oss-20b`

**20B (MoE)**

~3.5× smaller, open-weight

*Not a primary/fallback pair — both models process every claim, for comparison purposes; the winner informs the eventual single-model production choice.*

#### WHY NOT A PAID FRONTIER API (E.G. GPT-4O)?

Cost scales with evaluation volume, and the empirical finding (next slide) that context quality — not model choice — gates correctness weakens the case for a premium model.

#### WHY NOT SELF-HOSTED OPEN-SOURCE?

GPU hosting is incompatible with the CPU-basic HF Spaces deployment target. Groq's hosted API delivers the same open-weight-model benefits without the hosting burden.

**Selected for: free-tier, fast (0.7–0.9s/report), OpenAI-compatible API — swapping providers later is a base-URL change.**

# One Claim, Start to Finish

| | | |
|---|---|---|
| 1 | **Input** | Claimant uploads a damage photo, selects a policy from the catalog, optionally adds an incident narrative. |
| 2 | **Detect** | Damage Agent: letterbox → YOLO11m → NMS → class, confidence, bbox, area_ratio per instance. |
| 3 | **Score severity** | Severity Agent applies the global threshold table to each detection's area_ratio. |
| 4 | **Retrieve clauses** | Two-pass, doc-scoped coverage + exclusion queries per detected class (Section 8/9). |
| 5 | **Generate report** | Report Agent assembles the context bundle and returns a structured verdict per class, with citations and an escalation flag. |

**Escalation is not a stage-skipping gate: every claim reaches the Report Agent. Low confidence or missing coverage is surfaced as escalate_to_human inside the structured output.**

# 10 Contrastive Claims, Both Models, Mechanical Grading

Rather than a synthetic dry run, the Policy + Report Agent stack was verified against 10 deliberately contrastive scenarios — multi-class, escalation path, corpus-quirk stress tests — run through the real retrieval and generation stack, both models, scored by a mechanical (not LLM-judged) evaluator.

| | |
|---|---|
| **schema_valid** | Required keys present, verdict in the allowed set |
| **citation_validity** | Every cited clause was actually offered to the model |
| **verdict_evidence_consistent** | A "covered" verdict must cite ≥1 coverage chunk |
| **escalation_consistent** | Self-reported flag matches the payload's true state |
| **currency_violation** | No monetary figure invented outside the offered clauses |

**1.0**

**COMPOSITE SCORE — BOTH MODELS**

10 payloads × 2 models = 20 reports

Zero fabricated citations · Zero currency violations · Full grounding on every hard check

*Two data-quality issues were found and documented: a clause-tagging bug (fixed) and PDF table-row garbling in one policy (found, flagged for correction).*

# Context Quality Gates Correctness

### BEFORE THE FIX

Same claim, flawed retrieval context (a real coverage clause was filtered out by a tagging bug):

`llama-3.3-70b → "covered"`

`gpt-oss-20b → "excluded"`

*Two models, same input, opposite confident verdicts.*

### AFTER THE FIX

Same claim, corrected context — no change to prompt, temperature, or model:

`llama-3.3-70b → "covered"`

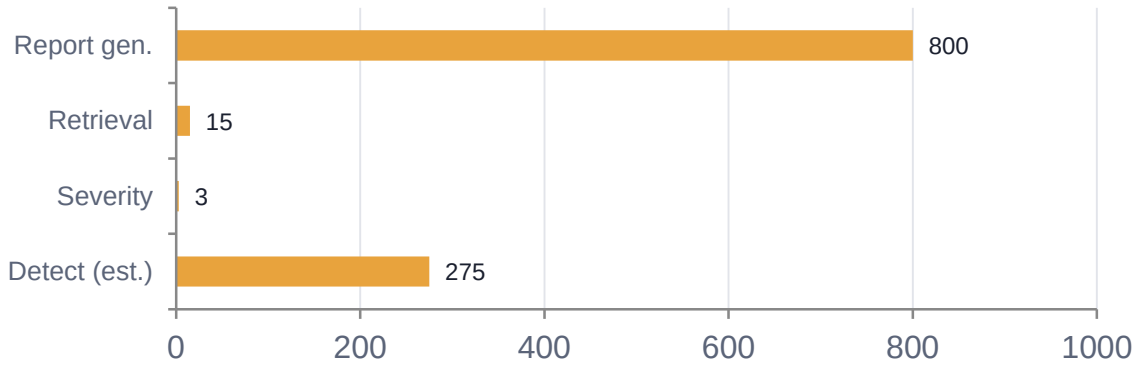
`gpt-oss-20b → "covered"`

*Both models converge on the same correct answer.*

# Estimated Latency

| Phase | Hardware | Measured cost |
|---|---|---|
| YOLO11m fine-tuning | T4 (Colab / Kaggle, 16 GB) | ~8.5–9.4 GB VRAM @ batch=4, 1280px |
| Embedding + retrieval | CPU only | Cold init ~16.5s; warm query ~10ms |
| Report generation | Groq API (remote) | ~0.7–0.9s per report round-trip |
| Deployed demo | HF Spaces CPU-basic (2 vCPU, 16 GB) | UI not yet built (Section 15) |

**PER-CLAIM LATENCY (NON-ESCALATED)**




**~1–1.5s**

total, single-class, non-escalated claim

*Report generation dominates latency; detection latency is estimated (no CPU inference measured yet).*

# Named Risks, With Their Current Mitigation Status

| | |
|---|---|
| **Class imbalance**<br>6.59:1 scratch:shattered_glass | Uniform cls-loss gain applied in probes; per-class weight vector not yet wired in |
| **Severity is global, not per-class**<br>Known bias by damage footprint size | Per-class calibration is identified future work — no data exists to calibrate against |
| **Domain shift**<br>Studio photos vs. real handheld claims | Partially addressed via augmentation; not yet stress-tested |
| **No orchestration framework**<br>Fixed sequential function calls | LangGraph wrapping planned for Milestone 4 |
| **PDF table-row garbling**<br>Confirmed in ≥1 policy document | Found via evaluation; fix flagged, not yet applied |

# Ready for Milestone 4

## ALREADY DONE THIS MILESTONE

- YOLO11m vs YOLOv8m empirical probe — architecture selected
- Doc-scoped hybrid retrieval reproduces its M2 evaluation exactly
- 10-payload faithfulness evaluation: 1.0 composite, both Groq models
- 315-case census formally rules out damage-based policy inference

## PLANNED FOR MILESTONE 4

- Full 50-epoch YOLO11m + YOLOv8m baseline runs, real mAP@50/F1
- Wire Milestone 2's per-class loss weights (replacing uniform gain)
- True end-to-end run with live YOLO inference, not synthetic payloads
- Calibrate the escalation confidence threshold against real scores
- Fix the PDF table-garbling issue; apply the clause-tag root-cause fix
- Wrap pipeline stages as LangGraph nodes if an orchestrator is needed